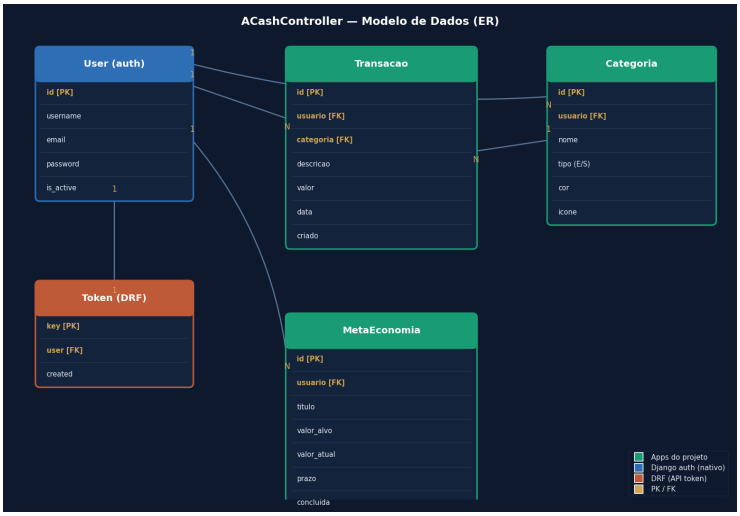
ACashController — Documentação do Trabalho

> Trabalho final de Django • Grupo de 3 • Entrega 11/06/2026 (30 pts projeto + 20 pts arguição)

Índice dos documentos

#	Arquivo	O que tem
00	00-estrutura-projeto.md	Estrutura de pastas espelhando o repo `ecommerce2026`
01	01-plano-execucao.md	Plano, milestones, divisão entre as 3 pessoas
02	02-modelagem-dados.md	Models, relacionamentos, regra de saldo, dados dos gráficos
		
03a	03-prompt-claude-design.md	Prompt pronto pro Claude Design (telas HTML)
03b	03-prompt-claude-code.md	Prompts pro Claude Code (backend + API), por milestone
04	04-documentacao-iterativa.md	Doc vivo: como o projeto funciona (pra arguição)
05	05-perguntas-professor.md	Banco de perguntas + respostas (os 20 pts)
06	06-deploy-e-variaveis.md	Deploy (PythonAnywhere) + variáveis de ambiente (.env)
07	07-mapa-telas-templates.md	Mapa telas (Claude Design) → templates Django + URL names + ids dos campos

#	Arquivo	O que tem
08	08-prompt-claude-design-faltantes.md	Prompt usado pra gerar as 3 telas faltantes (confirm_delete, meta_form, extrato_pdf)

Stack

Django 5.2 · SQLite · DRF + TokenAuthentication · Chart.js · xhtml2pdf · django-environ · GitHub Codespaces · Deploy: PythonAnywhere.

Ordem sugerida de leitura

01 → 02 (+ diagrama) → 00 → 07 → 03a → 03b → 04 → 05 → 06. (08 é referência do prompt de design.)

> **Status das telas: 12 telas completas** geradas no Claude Design + `styles.css` (design system: green #1D9E75, blue #2E6FB7, red/expense #C25E3A, fonte Poppins). HTMLs originais em `telas/`. Decisão do grupo: docs mandam na convenção de nomes Django — as telas são renomeadas pra bater (ver doc 07). Pacote pronto pra alimentar o **Claude Code** (doc 03b).

Estrutura do Projeto (espelhando ecommerce2026)

```
acashcontroller/
├── manage.py
├── requirements.txt
├── .env_vazio                # modelo de variáveis (igual ao repo de ref.)
├── README.md
│
├── core/                     # projeto Django (settings central)
│   ├── __init__.py
│   ├── settings.py           # INSTALLED_APPS, DRF, auth, static, templates
│   ├── urls.py               # include() de cada app + admin + api
│   ├── asgi.py
│   └── wsgi.py
│
├── utils/                    # mixins reutilizáveis (igual repo ref.)
│   └── my_mixins.py          # GeraPDFMixin, UsuarioQuerysetMixin
│
├── usuario/                  # auth: cadastro, login, logout
│   ├── views.py              # CBV (CreateView de cadastro)
│   ├── forms.py
│   ├── urls.py
│   ├── tests.py
│   └── templates/
│       ├── registration/
│       │   ├── login.html
│       │   └── logout.html
│       └── usuario/
│           └── cadusuario.html
│
├── financas/                 # núcleo: categorias, transações, metas
│   ├── models.py             # Categoria(tem tipo E/S), Transacao(tipo derivado), MetaEconomia
│   ├── views.py              # CBV CRUD + validação de saldo
│   ├── forms.py              # TransacaoForm (clean: valida saldo)
│   ├── admin.py
│   ├── urls.py
│   ├── managers.py           # managers customizados (opcional)
│   ├── tests.py              # teste de validação de saldo + isolamento
│   ├── migrations/
│   └── templates/financas/
│       ├── transacao_list.html
│       ├── transacao_form.html
│       ├── transacao_confirm_delete.html
│       ├── categoria_list.html
│       ├── categoria_form.html
│       ├── meta_list.html
│       └── meta_form.html
│
├── dashboard/                # agregações, gráficos, exportação
│   ├── views.py              # DashboardView, ExportarCSVView, ExportarPDFView
│   ├── urls.py
│   ├── tests.py
│   └── templates/dashboard/
│       ├── dashboard.html    # Chart.js (linha + pizza)
│       └── extrato_pdf.html  # template do PDF (xhtml2pdf)
│
├── api/                      # DRF + TokenAuthentication
│   ├── serializers.py
│   ├── views.py              # ViewSets (get_queryset por usuário)
│   ├── urls.py               # router + obtain_auth_token
│   └── tests.py              # teste de auth da API
│
```

```
├── templates/
│   └── base.html              # layout central, navbar, messages
│
└── static/
    ├── css/styles.css
    └── js/charts.js           # init do Chart.js consumindo o JSON do dashboard
```

Pontos de paralelo com o repo de referência

ecommerce2026	nosso projeto
`utils/my_mixins.py` (`GeraPDFMixin`)	mesmo arquivo, mesmo mixin para o PDF
`catalogo/models.py` (manager `disponiveis`)	`financas/managers.py` (filtro por usuário)
views 100% CBV	idem
`templates/<app>/` por app + `base.html`	idem
`urls.py` central com `include()`	idem
Django 5.2 + `i18n_patterns`	Django 5.2 (i18n opcional — pode dispensar)

requirements.txt (base)

```
Django==5.2
djangorestframework==3.15.2
xhtml2pdf==0.2.17
pillow==12.1.1
python-environ==0.4.54
```

> Chart.js entra por **CDN** no template — não é dependência Python.

Plano de Execução — ACashController

> **ACashController** — Aplicação de controle financeiro pessoal Trabalho final • Django • Grupo de 3 pessoas **Entrega:** 11/06/2026 — 30 pts (projeto) + 20 pts (perguntas orais)

---

1. Visão geral

Aplicação web Django para **controle de receitas, despesas e metas de economia**, com:

- Lançamento de transações categorizadas (Entradas/Saídas) via **CBV**, com **validação de saldo**.
- **Dashboard** com gráfico de linha (evolução mensal do patrimônio) + gráfico de pizza (gastos por categoria) usando **Chart.js**.
- Exportação de **extrato mensal em CSV e PDF**.
- **API REST** (DRF) protegida por **TokenAuthentication**.
- Isolamento por usuário: `LoginRequiredMixin` + filtragem em `get_queryset()`.

Stack fechada

Camada	Escolha
Framework	Django **5.2**
Banco	**SQLite**
API	**Django REST Framework** + TokenAuthentication
Gráficos	**Chart.js** (front, via CDN)
PDF	**xhtml2pdf** (pisa) — mesmo do repo de referência
Ambiente	**GitHub Codespaces** + repo novo no GitHub
Design	**Claude Design** (telas HTML) → **Claude Code** (backend/API)

Espelhando o repo de referência

Arquitetura igual a `github.com/marcos-faino/ecommerce2026`: - Projeto Django multi-app, **views 100% CBV**. - Mixins reutilizáveis em `utils/my_mixins.py` (ex.: `GeraPDFMixin`). - Cada app com seus próprios `templates/<app>/`, `base.html` central. - `settings.py` central, URLs incluídas por app via `include()`.

---

2. Divisão de apps Django

App	Responsabilidade
`usuario`	Cadastro, login/logout, perfil. Reaproveita auth nativo.
`financas`	Núcleo: models `Categoria`, `Transacao`, `MetaEconomia`, CBVs de CRUD + validação de saldo.
`dashboard`	Views agregadoras: cálculo de saldo, séries mensais, dados pros gráficos, exportação CSV/PDF.

App	Responsabilidade
`api`	DRF: serializers, viewsets, rotas, TokenAuthentication.
`utils`	Mixins compartilhados (`GeraPDFMixin` , `UsuarioQuerysetMixin`).

> Pode-se mesclar `dashboard` dentro de `financas` se o professor preferir menos apps. Mantemos separado para clareza de responsabilidades (alinha com SOLID/SRP — bom argumento na arguição).

---

3. Milestones (com checklist)

M0 — Setup (Dia 1)

- [ ] Criar repo novo no GitHub + abrir Codespace.
- [ ] `python -m venv venv` / `requirements.txt` .
- [ ] `django-admin startproject core .`
- [ ] Criar apps: `usuario` , `financas` , `dashboard` , `api` , pasta `utils` .
- [ ] Registrar apps + `rest_framework` + `rest_framework.authtoken` em `INSTALLED_APPS` .
- [ ] `.gitignore` (.env, db.sqlite3, staticfiles) + `.env.example` .
- [ ] Primeiro commit + push na `main` .

M1 — Modelagem & banco (Dia 1-2)

- [ ] Models `Categoria` , `Transacao` , `MetaEconomia` (ver `02-modelagem-dados.md`).
- [ ] Manager customizado por usuário (espelha `ProdutosDisponiveisManager`).
- [ ] `makemigrations` + `migrate` .
- [ ] Registrar no `admin.py` de cada app.
- [ ] Criar superuser + dados de teste (categorias: alimentação, moradia, assinaturas, salário, saúde…).

M2 — Auth & isolamento por usuário (Dia 2)

- [ ] Views de login/logout (auth nativo) + cadastro (`CreateView`).
- [ ] `LoginRequiredMixin` em todas as views privadas.
- [ ] `UsuarioQuerysetMixin` filtrando `get_queryset()` por `request.user` .
- [ ] `form_valid()` setando `usuario = request.user` no save.

M3 — CRUD de transações + validação de saldo (Dia 2-3)

- [ ] `TransacaoListView` , `CreateView` , `UpdateView` , `DeleteView` .
- [ ] CRUD de `Categoria` e `MetaEconomia` .
- [ ] **Validação de saldo**: ao lançar Saída, recusar se saldo insuficiente (no `form.clean()` ou `form_valid()`), com `messages.error` .

M4 — Dashboard + gráficos (Dia 3-4)

- [ ] `DashboardView (TemplateView)` calculando saldo, totais e séries.
- [ ] Agregações com `TruncMonth` + `Sum` (evolução mensal) e `values('categoria').annotate(Sum)` (pizza).
- [ ] Endpoints JSON (ou contexto serializado) consumidos pelo Chart.js.
- [ ] Front: Chart.js (linha + pizza) via CDN.

M5 — Exportação CSV/PDF (Dia 4)

- [ ] Extrato CSV (`csv` + `HttpResponse content_type=text/csv`).
- [ ] Extrato PDF reaproveitando `GeraPDFMixin` (xhtml2pdf) — template `extrato_pdf.html` .
- [ ] Filtro por mês/ano.

M6 — API DRF + Token (Dia 5)

- [ ] Serializers de `Transacao` / `Categoria` / `MetaEconomia` .
- [ ] ViewSets com `permission_classes=[IsAuthenticated]` e `authentication_classes=[TokenAuthentication]` .
- [ ] `get_queryset()` filtrando por `self.request.user` .
- [ ] Endpoint `obtain_auth_token` para emitir token.
- [ ] Testar com `curl` /Postman.

M7 — Telas (Claude Design) + polish (Dia 5-6)

- [ ] Gerar HTML das telas no Claude Design → integrar nos templates.
- [ ] `base.html` , navbar, mensagens, responsivo.
- [ ] Revisar UX do dashboard.

M9 — Variáveis de ambiente & deploy (Dia 7) — opcional

- [ ] `django-environ` + `.env` / `.env.example` (segredos fora do Git).
- [ ] `DATABASES` lendo `DATABASE_URL` ; `STATIC_ROOT` + `collectstatic` .
- [ ] Deploy no **PythonAnywhere** (free, SQLite persistente). Ver `06-deploy-e-variaveis.md` .
- [ ] `DEBUG=False` + `ALLOWED_HOSTS` em produção.

M8 — Testes, docs & ensaio da arguição (Dia 6-7)

- [ ] Testes básicos (`tests.py`): validação de saldo, isolamento de queryset, auth da API.
- [ ] Fechar a documentação iterativa (`04-documentacao-iterativa.md`).
- [ ] Cada integrante domina sua parte + simulação de perguntas (`05-perguntas-professor.md`).
- [ ] README com instruções de rodar.

---

4. Divisão entre as 3 pessoas (sugestão)

Pessoa	Frente	Apps/arquivos
Dev A	Modelos + CRUD + validação de saldo	`financas/models.py` , `financas/views.py` , forms
Dev B	Dashboard + gráficos + exportação CSV/PDF	`dashboard/` , `utils/my_mixins.py` , templates de gráfico
Dev C	API DRF + Token + Auth/isolamento + testes	`api/` , `usuario/` , `tests.py`

> Todos revisam o código dos outros (cada um precisa saber explicar **tudo** na arguição — ver doc 05).

---

5. Fluxo de trabalho com Claude Design + Claude Code

```
Claude Design  →  telas HTML simples (base, dashboard, lista, form, extrato)
        │
        ▼
Claude Code    →  backend Django (models, CBVs, mixins) + API DRF + Chart.js wiring
        │
```

```
        ▼
GitHub Codespaces  →  rodar, testar, commitar (push main)
```

1. **Claude Design**: usar o prompt do doc `03-prompt-claude-design.md` para gerar o HTML estático das telas.
2. **Claude Code**: usar o prompt do doc `03-prompt-claude-code.md` para implementar o backend e ligar os dados aos templates/gráficos.
3. Sempre commitar por milestone. Mensagens claras (servem de prova de autoria na arguição).

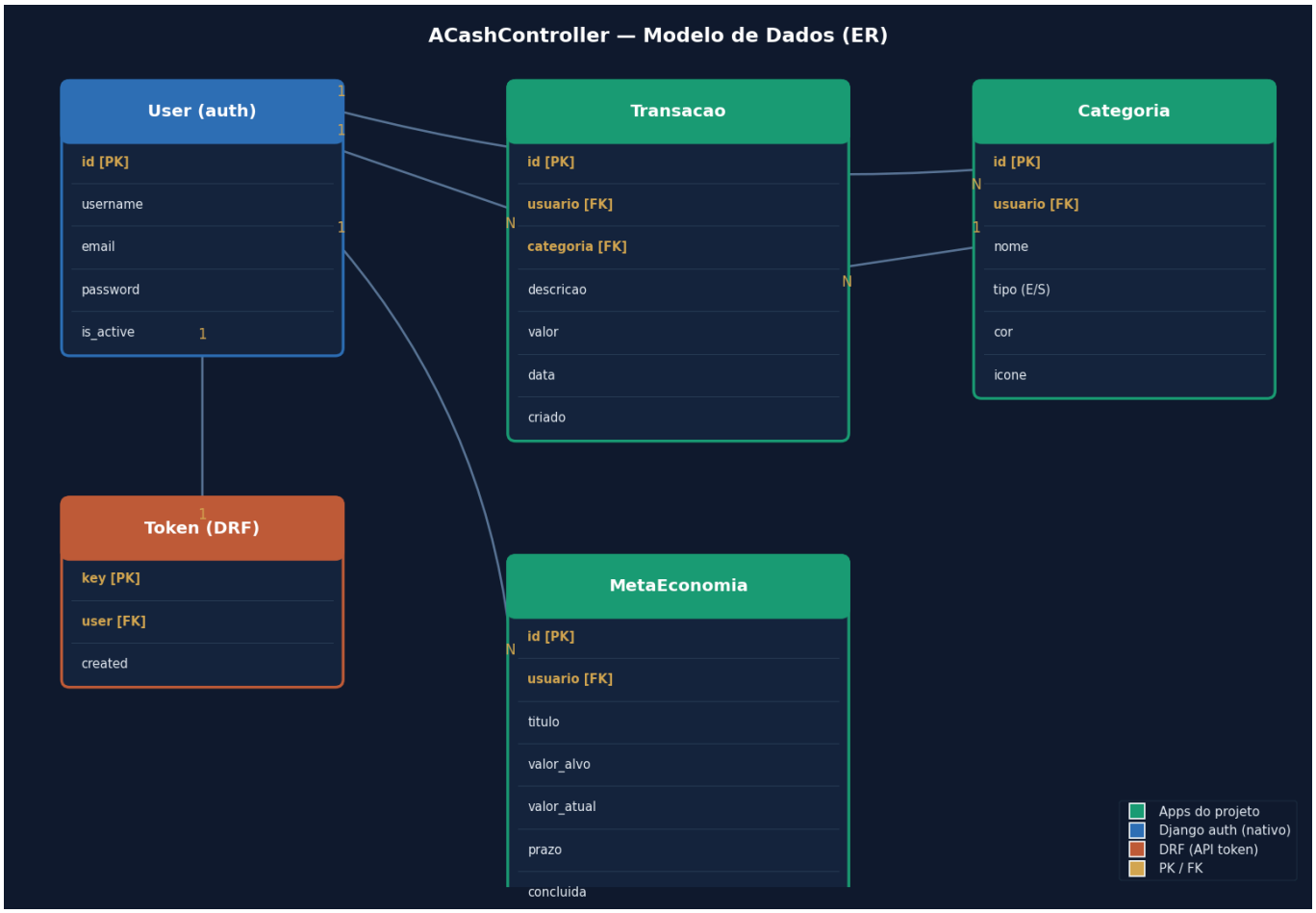
---

6. Critérios de avaliação → onde cada ponto é coberto

Requisito do enunciado	Onde é atendido	Milestone
CBV para lançamento de transações categorizadas	`financas/views.py` (CreateView etc.)	M3
Validação de saldo suficiente	`form.clean()` / `form_valid()`	M3
Dashboard linha (patrimônio mensal)	`dashboard` + Chart.js	M4
Pizza (gastos por categoria)	`dashboard` + Chart.js	M4
Exportação CSV ou PDF	CSV + `GeraPDFMixin`	M5
API consumível de transações	`api/` DRF ViewSet	M6
Token Authentication	DRF `TokenAuthentication`	M6
Cada usuário só vê seus dados	`LoginRequiredMixin` + `get_queryset`	M2

Cobrimos **100%** do enunciado, com folga (PDF **e** CSV).

Modelagem de Dados — ACashController



Resumo das entidades

Entidade	App	Papel
`User`	`django.contrib.auth` (nativo)	Dono de todos os dados. Não criamos — usamos o de Django.
`Token`	`rest_framework.authtoken`	1:1 com User. Autentica chamadas da API.
`Categoria`	`financas`	Classifica transações (Entrada ou Saída). Por usuário.
`Transacao`	`financas`	Lançamento financeiro (E/S), valor, data, categoria. Por usuário.
`MetaEconomia`	`financas`	Meta de economia com valor-alvo e progresso. Por usuário.

Relacionamentos

- `User 1 — N Categoria`
- `User 1 — N Transacao`
- `User 1 — N MetaEconomia`
- `Categoria 1 — N Transacao`

- `User 1 — 1 Token` (DRF)

> **Decisão de modelagem (importante para a arguição):** o campo **`tipo` (Entrada/Saída) fica APENAS na `Categoria`**, não na `Transacao`. Como toda transação aponta para uma categoria (1:N), o tipo do lançamento é **derivado** de `transacao.categoria.tipo`. Guardar `tipo` também na `Transacao` seria **redundância** e poderia gerar **inconsistência** (uma transação 'S' apontando para uma categoria 'E'). Por isso `Transacao` expõe `tipo` / `tipo_display` como `@property` que leem a categoria — sem coluna duplicada no banco (modelo normalizado).

Tudo amarrado ao `User` → garante o isolamento por usuário exigido no enunciado.

Models (Django) — referência de implementação

> Espelha o estilo do repo de referência: managers customizados, `Meta`, `__str__`, `verbose_name`.

```
# financas/models.py
from django.db import models
from django.contrib.auth.models import User
from django.core.exceptions import ValidationError


class TipoTransacao(models.TextChoices):
    ENTRADA = 'E', 'Entrada'
    SAIDA = 'S', 'Saída'


class Categoria(models.Model):
    usuario = models.ForeignKey(User, on_delete=models.CASCADE,
                                related_name='categorias')
    nome = models.CharField(max_length=100)                   # tela: maxlength=100
    tipo = models.CharField(max_length=1, choices=TipoTransacao.choices)
    cor = models.CharField(max_length=7, default='#1D9E75')   # hex p/ Chart.js (input type=color)
    icone = models.CharField(max_length=50, blank=True)       # tela: maxlength=50, opcional (ex.: cart, home)

    class Meta:
        verbose_name = 'Categoria'
        verbose_name_plural = 'Categorias'
        ordering = ['nome']
        unique_together = ('usuario', 'nome', 'tipo')

    def __str__(self):
        return f'{self.nome} ({self.get_tipo_display()})'


class Transacao(models.Model):
    usuario = models.ForeignKey(User, on_delete=models.CASCADE,
                                related_name='transacoes')
    categoria = models.ForeignKey(Categoria, on_delete=models.PROTECT,
                                  related_name='transacoes')
    descricao = models.CharField(max_length=160)
    valor = models.DecimalField(max_digits=12, decimal_places=2)
    data = models.DateField()
    criado = models.DateTimeField(auto_now_add=True)

    class Meta:
        verbose_name = 'Transação'
        verbose_name_plural = 'Transações'
        ordering = ['-data', '-criado']
```

```
    def __str__(self):
        return f'{self.tipo_display} | {self.descricao} | R$ {self.valor}'

    @property
    def tipo(self):
        # Tipo é DERIVADO da categoria — não há campo duplicado em Transacao.
        return self.categoria.tipo

    @property
    def tipo_display(self):
        return self.categoria.get_tipo_display()

    def clean(self):
        if self.valor is None or self.valor <= 0:
            raise ValidationError({'valor': 'O valor deve ser maior que zero.'})


class MetaEconomia(models.Model):
    usuario = models.ForeignKey(User, on_delete=models.CASCADE,
                                related_name='metas')
    nome = models.CharField(max_length=120)                   # tela usa .goal-name = "nome"
    valor_alvo = models.DecimalField(max_digits=12, decimal_places=2)
    valor_atual = models.DecimalField(max_digits=12, decimal_places=2, default=0)
    prazo = models.DateField(null=True, blank=True)           # tela: "Prazo: dd/mm/yyyy"
    concluida = models.BooleanField(default=False)

    class Meta:
        verbose_name = 'Meta de Economia'
        verbose_name_plural = 'Metas de Economia'
        ordering = ['concluida', 'prazo']

    def __str__(self):
        return self.nome

    @property
    def progresso(self):
        # usado no template: width do .goal-fill e texto "{{ progresso }}% concluído"
        if not self.valor_alvo:
            return 0
        return min(100, round((self.valor_atual / self.valor_alvo) * 100, 1))
```

Regra de negócio: cálculo de saldo e validação

O **saldo** não é armazenado — é **derivado** das transações (fonte única de verdade, evita inconsistência):

```
# saldo = soma(Entradas) - soma(Saídas) do usuário
from django.db.models import Sum, Q

def saldo_usuario(user):
    entradas = user.transacoes.filter(categoria__tipo='E').aggregate(s=Sum('valor'))['s'] or 0
    saidas   = user.transacoes.filter(categoria__tipo='S').aggregate(s=Sum('valor'))['s'] or 0
    return entradas - saidas
```

Validação ao lançar uma Saída (no form ou na view):

```
# financas/forms.py — dentro do clean()
def clean(self):
    cleaned = super().clean()
    categoria = cleaned.get('categoria')
    valor = cleaned.get('valor')
    # O tipo (E/S) NÃO está na transação: é derivado da categoria escolhida.
```

```
    if categoria and valor and categoria.tipo == 'S':
        if saldo_usuario(self.usuario) < valor:
            raise ValidationError('Saldo insuficiente para esta saída.')
    return cleaned
```

> O `self.usuario` é injetado pela view via `get_form_kwargs()`. Esse é o ponto que o professor provavelmente vai cobrar — todos devem saber explicar **onde** e **por que** a validação acontece no form e não no model.

Categorias de exemplo (seed)

> Alinhado às telas (`categorias.html` / `transacoes.html` / dashboard "Top Categorias"). `cor` em hex porque alimenta o Chart.js e o swatch da listagem.

Nome	Tipo	Cor (hex)	Ícone
Salário	Entrada	#1D9E75	briefcase
Freelance	Entrada	#2E6FB7	laptop
Alimentação	Saída	#C25E3A	cart
Moradia	Saída	#7C3AED	home
Assinaturas	Saída	#D97706	—
Saúde	Saída	#DC2626	heart-pulse
Transporte	Saída	#2E6FB7	—
Lazer	Saída	#8B5CF6	—

Dados para os gráficos (context vars EXATAS das telas)

A `dashboard.html` espera estes nomes no contexto da `DashboardView` (não renomear — as telas já referenciam):

Context var	Tipo	Onde é usada na tela
`saldo`	Decimal	`.stat-card.blue` → `R$ {{ saldo|floatformat:2 }}`
`total_entradas`	Decimal	`.stat-card.green`
`total_saidas`	Decimal	`.stat-card.red`
`mes_nome`, `ano`	str/int	`.stat-sub` → "Junho 2026"
`patrimonio_labels`	list (JSON)	gráfico de **linha** `canvas#chartPatrimonio`
`patrimonio_data`	list (JSON)	idem
`cat_labels`	list (JSON)	gráfico de **rosca** `canvas#chartCategorias`
`cat_data`	list (JSON)	idem

Context var	Tipo	Onde é usada na tela
`top_categorias`	list de dicts	barras de progresso "Top Categorias" (`{nome, pct}`)

> No template os JSON entram com `{{ patrimonio_labels\|safe }}` etc. (ou via `json_script`). Chart.js **4.4.0** via CDN (`chart.umd.min.js`). O `cutout: '62%'` faz a rosca.

Linha (evolução mensal do patrimônio):

```
from django.db.models.functions import TruncMonth
from django.db.models import Sum

(Transacao.objects.filter(usuario=user)
    .annotate(mes=TruncMonth('data'))
    .values('mes', 'categoria__tipo')
    .annotate(total=Sum('valor'))
    .order_by('mes'))
# no Python: acumular (entradas - saídas) mês a mês = patrimônio acumulado
# -> alimenta patrimonio_labels (['Jan','Fev',...]) e patrimonio_data ([12000,...])
```

Rosca/pizza (gastos por categoria) + Top Categorias:

```
gastos = (Transacao.objects.filter(usuario=user, categoria__tipo='S')
    .values('categoria__nome', 'categoria__cor')
    .annotate(total=Sum('valor'))
    .order_by('-total'))
# cat_labels  = [g['categoria__nome'] for g in gastos]
# cat_data    = [g['total'] for g in gastos]   (ou % do total de saídas)
# top_categorias = mesmos dados convertidos em [{'nome':..., 'pct':...}]  (5 maiores)
```

Esses resultados viram JSON e alimentam o Chart.js + as barras de progresso.

Mapa: Telas (Claude Design) → Templates Django + URLs

> **Decisão do grupo:** os docs mandam na convenção de nomes (Django). As telas geradas pelo Claude Design serão **renomeadas/adaptadas** para bater com esta tabela — NÃO o contrário. Cada arquivo HTML entregue já traz comentários `<!-- Django: ... -->` com os `{% url %}`, `{% block %}` e variáveis de contexto. Use-os como guia ao integrar.

---

1. Renomeação dos arquivos HTML → templates

Arquivo entregue (Claude Design)	Template Django (destino)	App / pasta
`base.html`	`templates/base.html`	projeto (central)
`login.html`	`usuario/templates/registration/login.html`	usuario
`cadastro.html`	`usuario/templates/usuario/cadusuario.html`	usuario
`dashboard.html`	`dashboard/templates/dashboard/dashboard.html`	dashboard
`transacoes.html`	`financas/templates/financas/transacao_list.html`	financas
`transacao_form.html`	`financas/templates/financas/transacao_form.html`	financas
`categorias.html`	`financas/templates/financas/categoria_list.html`	financas
`categoria_form.html`	`financas/templates/financas/categoria_form.html`	financas
`metas.html`	`financas/templates/financas/meta_list.html`	financas
`transacao_confirm_delete.html`	`financas/templates/financas/transacao_confirm_delete.html`	financas
`meta_form.html`	`financas/templates/financas/meta_form.html`	financas
`extrato_pdf.html`	`dashboard/templates/dashboard/extrato_pdf.html`	dashboard
`styles.css`	`static/css/styles.css`	projeto

> **12 telas geradas (completas).** As 3 últimas (`transacao_confirm_delete` , `meta_form` , `extrato_pdf`) já vieram consistentes com o design system. Originais ficam em `telas/` no repo.
>
> O CSS já tem classes para tudo: `.filter-bar` , `.pagination` , `.page-btn` , `.goals-grid` , `.goal-card` , `.dashboard-split` , `.chart-card` , `.charts-bottom` , `.cat-row/.progress-fill` , `.color-swatch` , `.badge-entrada/-saida` , `.stat-card.green/.blue/.red` . Não inventar classes novas.

---

2. URLs esperadas pelas telas (já nos comentários `Django:`)

Crie estes `name=` nos `urls.py` . A navbar (`base.html`) e os botões dependem deles.

usuario

name	view	nota
`login`	LoginView	`registration/login.html`
`logout`	LogoutView	link "Sair" na navbar
`cadastro`	CreateView (UserCreationForm)	template `cadusuario.html`

financas — Transação

name	view	usado em
`transacoes`	TransacaoListView	navbar, filtro `<form action>`
`transacao_nova`	TransacaoCreateView	botão "+ Nova Transação"
`transacao_editar <pk>`	TransacaoUpdateView	ícone ✎ na linha
`transacao_excluir <pk>`	TransacaoDeleteView (POST)	ícone ✕ na linha

> `transacao_form.html` usa `<form method="post">` único pra criar e editar (`{% if form.instance.pk %}`). Os docs antigos citavam `transacao_salvar` — **substituído** por `transacao_nova` + `transacao_editar`.

financas — Categoria

name	view	usado em
`categoria_list`	CategoriaListView	navbar (hoje está `href="#"` → trocar)
`categoria_nova`	CategoriaCreateView	botão "+ Nova Categoria"
`categoria_editar <pk>`	CategoriaUpdateView	ícone ✎
`categoria_excluir <pk>`	CategoriaDeleteView (POST)	ícone ✕
`categoria_salvar`	(action do form)	a tela usa `action="{% url 'categoria_salvar' %}"` — pode apontar pro mesmo create/update ou trocar o action por `transacao_form`-style. **Recomendado:** trocar o `action` da tela para `categoria_nova` / `categoria_editar` e dispensar `categoria_salvar`.

financas — Meta

name	view	usado em
`meta_nova`	MetaCreateView	botão "+ Nova Meta" (dashboard + metas)
`meta_editar <pk>`	MetaUpdateView	botão "Editar" no card
`meta_excluir <pk>`	MetaDeleteView (POST)	botão "Excluir" no card

> `meta_list.html` (metas.html) não tem botão de listagem na navbar com url própria além de `metas` — adicionar

> `name='metas'` na ListView se quiser link direto; a navbar aponta pra `metas.html`.

dashboard

name	view	usado em
`dashboard`	DashboardView (TemplateView)	navbar brand + link
`exportar_csv`	ExportarCSVView	botão "↓ Exportar CSV" (sidebar)
`exportar_pdf`	ExportarPDFView (GeraPDFMixin)	botão "↓ Exportar PDF" (sidebar)

---

3. Campos de formulário (ids = `id_<campo>`) por tela

> Django gera `id_<nome_do_campo>` automaticamente. As telas já usam esses ids — manter os mesmos `name=` no Form garante o casamento.

login.html → `id_username`, `id_password` **cadusuario.html** (cadastro) → `id_username`, `id_email`, `id_password1`, `id_password2` (UserCreationForm) **transacao_form.html** → `id_descricao`, `id_valor`, `id_data`, `id_categoria` - `id_categoria` é um `<select>` com `<optgroup label="Saídas">` / `<optgroup label="Entradas">`. - **NÃO há campo `tipo`** — hint na tela: "o tipo é definido pela categoria". Consistente com o model. **categoria_form.html** → `id_nome`, `id_tipo` (select E/S), `id_cor` (`<input type="color">`), `id_icone` (opcional) - JS da tela só faz preview do swatch e do badge — a validação real é no `form.clean()`. **meta_form.html** → `id_nome`, `id_valor_alvo` (number step 0.01), `id_valor_atual` (number, default 0), `id_prazo` (date, opcional) - `valor_alvo` + `valor_atual` ficam num `.form-row` (2 colunas). Campo é **`nome`** (não titulo). - JS opcional faz preview do progresso reusando `.goal-track` / `.goal-fill`. `action` condicional `meta_nova` / `meta_editar`.

transacao_confirm_delete.html → sem form fields; usa `object.*` (padrão `DeleteView`): `{{ object.descricao }}`, `{{ object.categoria.nome }}`, `{{ object.data|date:"d/m/Y" }}`, `{{ object.valor|floatformat:2 }}`, tipo via `{% if object.categoria.tipo == 'E' %}` (lembrando: `tipo` é `@property`). POST form → `{% url 'transacao_excluir' object.pk %}` + `{% csrf_token %}`, botões `.btn-danger` (Excluir) + `.btn-outline` (Cancelar). Reusar este mesmo padrão para `categoria_confirm_delete` e `meta_confirm_delete` se preferir confirmação dedicada (as telas de categoria/meta usam botão ✕/Excluir direto — pode ser POST inline OU redirecionar pra esta tela de confirmação).

extrato_pdf.html (renderizado por xhtml2pdf/pisa — NÃO é tela de browser) → standalone, **sem** base.html/navbar/JS/CDN, layout 100% em `<table>` com unidades `pt`. Context vars: `mes_nome`, `ano`, `saldo`, `total_entradas`, `total_saidas`, `transacoes` (loop com `{% empty %}`), `gerado_em` (`{{ gerado_em|date:"d/m/Y H:i" }}`), `user`. Classes próprias do PDF (`.badge-pdf`, `.val-saida-pdf` etc.) — NÃO dependem do styles.css. Render: `pisa.CreatePDF(render_to_string('dashboard/extrato_pdf.html', ctx), dest=response)`.

---

4. Erros de validação

As telas mostram erro via `.alert-inline` (`#erroValidacao`) e `.form-hint` em vermelho por campo:

```
{% if form.errors %}
  <div class="alert-inline"><span>⚠</span><span>Corrija os erros abaixo antes de salvar.</span></div>
{% endif %}
```

```
...
{% if form.valor.errors %}<span class="form-hint" style="color:var(--red);">{{ form.valor.errors|join:", " }}</span>{% endif %}
```

A mensagem de **"Saldo insuficiente"** (regra de negócio) cai aqui como erro de form (`non_field_errors` ou erro no campo `valor`).

---

5. Flash messages (toda tela)

`base.html` já tem o bloco. As classes batem com `messages` :

```
{% if messages %}{% for m in messages %}
  <div class="alert-container"><div class="alert alert-{{ m.tags }}">{{ m }}</div></div>
{% endfor %}{% endif %}
```

`m.tags` → `success` / `error` / `info` (classes `.alert-success/.alert-error/.alert-info` existem no CSS).

Prompt — Claude Design (telas HTML)

> Cole no Claude Design. Gera o HTML estático das telas; depois o Claude Code integra com Django.

```
You are designing the HTML/CSS screens for a Django academic project:
a Personal Finance Control Dashboard called ACashController (single user, in Brazilian Portuguese).

Goal: clean, simple, responsive static HTML screens (HTML + plain CSS, no
framework required, but Bootstrap 5 via CDN is acceptable). These will later be
wired to Django templates ({% block %}, {{ variables }}, {% for %}), so keep the
markup semantic and easy to templatize. All visible text in pt-BR.

Design language:
- Modern, light, professional fintech look. Generous whitespace.
- Primary color #1D9E75 (green), accent #2E6FB7 (blue), warning/expense #C25E3A.
- Cards with subtle shadow, rounded corners. Sans-serif (e.g. Poppins via CDN).
- Money in BRL (R$). Entradas in green, Saídas in red.

Produce these 9 screens as separate HTML files, each extending a shared base
layout with a top navbar (Dashboard, Transações, Categorias, Metas, Sair):

1. base.html — navbar + content area + flash message area (placeholders).
2. login.html — login form (usuário, senha) + link para cadastro.
3. cadastro.html — registration form (usuário, e-mail, senha, confirmar senha).
4. dashboard.html — the main screen:
   - 3 summary cards on top: Saldo atual, Total de Entradas, Total de Saídas (mês).
   - a LINE chart placeholder (canvas#chartPatrimonio) "Evolução do Patrimônio".
   - a PIE chart placeholder (canvas#chartCategorias) "Gastos por Categoria".
   - Include Chart.js via CDN and a <script> with SAMPLE data so the charts render
     in the static preview (Django will later replace the data via JSON).
   - buttons: "Exportar CSV" and "Exportar PDF".
5. transacoes.html — table of transactions (data, descrição, categoria, tipo,
   valor) with a "Nova Transação" button; filter by month/year; edit/delete icons.
   NOTE: the "tipo" column is display-only (Entrada/Saída) and is derived from the
   transaction's categoria — color the row/badge green for Entrada, red for Saída.
6. transacao_form.html — form to create/edit a transaction (descrição, valor,
   categoria [select], data). NOTE: there is NO separate Entrada/Saída field — the
   type comes from the chosen categoria. Show an inline error area for "Saldo
   insuficiente".
7. categorias.html — list of categories as a table/cards (nome, tipo [Entrada/Saída
   badge, green/red], a color swatch from `cor`, icon) with a "Nova Categoria" button
   and edit/delete icons.
8. categoria_form.html — form to create/edit a category: nome (text), tipo (select
   Entrada/Saída), cor (color picker / hex input), icone (optional text). This is the
   screen where the Entrada/Saída type is actually defined — transactions inherit it.
9. metas.html — list of saving goals as cards with a progress bar (valor_atual /
   valor_alvo), prazo, and a "Nova Meta" button.

Constraints:
- Keep CSS in a single <style> or one styles.css; avoid heavy JS beyond Chart.js.
- Use placeholder/sample data so every screen looks complete in preview.
- Mark obvious dynamic spots with HTML comments like <!-- {% for t in transacoes %} -->
  so the Django integration is easy.
- Mobile responsive (cards stack, table scrolls horizontally).

Deliver each screen as a complete standalone HTML file.
```

Depois de gerar

1. Baixe os HTML.
2. Coloque o layout em `templates/base.html` e cada tela em `templates/<app>/`.
3. O Claude Code (próximo doc) converte os placeholders em tags Django reais.

Prompt — Claude Code (backend + API)

> Cole no Claude Code, dentro do Codespace com os HTML do Claude Design já presentes. Use de forma iterativa: peça **um milestone por vez** (M1, depois M2…), revisando o código.

Prompt mestre (contexto inicial — cole uma vez)

```
We are building a Django 5.2 academic project: a "Dashboard de Controle Financeiro
(a Personal Finance Dashboard called ACashController) for a 3-person group. UI text in pt-BR.
Database: SQLite. API: Django REST Framework with TokenAuthentication.
Charts: Chart.js (CDN, front-end only). PDF export: xhtml2pdf (pisa).

ARCHITECTURE (mirror the reference repo github.com/marcos-faino/ecommerce2026):
- 100% Class-Based Views (CBV). No function views.
- Reusable mixins in utils/my_mixins.py (e.g. GeraPDFMixin from the reference,
  plus a UsuarioQuerysetMixin that filters get_queryset by request.user and sets
  usuario = request.user on form_valid).
- Per-app templates folders + a central templates/base.html.
- Central core/urls.py using include() per app.
- Managers/Meta/__str__ on models, like the reference repo.

APPS:
- usuario  : registration (CreateView) + login/logout (Django auth views).
- financas : models Categoria, Transacao, MetaEconomia; CBV CRUD; balance validation.
- dashboard: aggregation views, Chart.js data, CSV + PDF export.
- api      : DRF serializers, viewsets, token auth, router.
- utils    : shared mixins.

MODELS (financas/models.py):
- Categoria(usuario FK, nome, tipo[E/S], cor hex, icone).
- Transacao(usuario FK, categoria FK, descricao, valor Decimal, data, criado).
  NOTE: Transacao has NO `tipo` field — the Entrada/Saída type is DERIVED from
  categoria.tipo (expose `tipo` and `tipo_display` as @property reading the
  category). clean() rejects valor <= 0.
- MetaEconomia(usuario FK, nome, valor_alvo, valor_atual, prazo, concluida) with
  a `progresso` property (used by the .goal-fill width and "% concluído" text).

TEMPLATES: the HTML screens from Claude Design already exist in the repo. Wire them
1:1 — do NOT redesign. Rename the files to Django convention and keep the CSS classes
and field ids as-is. See docs/07-mapa-telas-templates.md for the full screen→template
→url-name mapping. Generated `id_<field>` must match the form field names.

KEY BUSINESS RULES (the professor WILL ask about these — implement clearly):
1. Balance is DERIVED, never stored: saldo = sum(Entradas) - sum(Saídas) for the user, filtering by categoria__tipo.
2. Balance validation: when creating a SAÍDA, reject in TransacaoForm.clean() if
   saldo_usuario(user) < valor, raising ValidationError("Saldo insuficiente...").
   The type comes from the chosen categoria (categoria.tipo == 'S'), NOT from a
   field on Transacao. The form receives `usuario` via the view's get_form_kwargs().
3. Per-user isolation EVERYWHERE: LoginRequiredMixin on all private views +
   get_queryset() filtered by request.user. Same in DRF viewsets.

Always: keep code readable, in pt-BR for user-facing strings, with comments on the
tricky parts. Ask me before large refactors. Implement ONE milestone at a time when
I ask. Confirm you understood this context and wait for my first milestone request.
```

Prompts por milestone (peça um de cada vez)

M1 — modelos & admin

```
Implement financas/models.py exactly as specified (Categoria[nome max_length=100,
tipo E/S, cor hex default #1D9E75, icone max_length=50 blank], Transacao,
MetaEconomia[nome, valor_alvo, valor_atual, prazo, concluida] with choices, Meta,
__str__, progresso property). Register all in financas/admin.py. Add a helper
saldo_usuario(user) in financas/services.py using aggregate(Sum) filtered by
categoria__tipo. Generate migrations. Seed example categories with colors/icons:
Salário(#1D9E75,briefcase) Freelance(#2E6FB7,laptop) [Entrada]; Alimentação
(#C25E3A,cart) Moradia(#7C3AED,home) Assinaturas(#D97706) Saúde(#DC2626,heart-pulse)
Transporte(#2E6FB7) Lazer(#8B5CF6) [Saída].
```

M2 — auth & isolamento

```
Implement usuario app: registration CreateView + login/logout via Django auth.
Create utils/my_mixins.py with UsuarioQuerysetMixin (filters get_queryset by
request.user; sets instance.usuario = request.user in form_valid). Keep the
GeraPDFMixin from the reference repo (xhtml2pdf). Wire URLs and LOGIN_URL.
```

M3 — CRUD transações + validação de saldo

```
Implement financas CBV CRUD for Transacao, Categoria, MetaEconomia using
ListView/CreateView/UpdateView/DeleteView + LoginRequiredMixin +
UsuarioQuerysetMixin. Build TransacaoForm with clean() that rejects a SAÍDA when
balance is insufficient (uses saldo_usuario). Pass `usuario` via get_form_kwargs().
URL names (used by the screens): transacoes, transacao_nova, transacao_editar<pk>,
transacao_excluir<pk>; categoria_list, categoria_nova, categoria_editar<pk>,
categoria_excluir<pk>; meta_nova, meta_editar<pk>, meta_excluir<pk>. The transacao
list has a GET filter form (mes/ano/q) + pagination (.pagination/.page-btn) — use
Django Paginator (page_obj). Integrate the existing HTML screens 1:1 (see doc 07).
```

M4 — dashboard + Chart.js

```
Implement dashboard.views.DashboardView (TemplateView) feeding dashboard.html.
The template ALREADY exists and expects EXACTLY these context vars (do not rename):
  saldo, total_entradas, total_saidas, mes_nome, ano,
  patrimonio_labels, patrimonio_data, cat_labels, cat_data, top_categorias.
- saldo derived (sum Entradas - sum Saídas, filter categoria__tipo).
- patrimonio_*: monthly accumulated net worth (TruncMonth + Sum).
- cat_*: expenses by category (values+annotate Sum), feed the doughnut (#chartCategorias).
- top_categorias: list of {nome, pct} for the "Top Categorias" progress bars (5 max).
Charts are Chart.js 4.4.0 (CDN) — the JS is inline in dashboard.html with canvas ids
chartPatrimonio (line) and chartCategorias (doughnut, cutout 62%). Just replace the
example arrays with {{ patrimonio_labels|safe }} etc. Use Categoria.cor for colors.
```

M5 — exportação CSV + PDF

```
Add dashboard ExportarCSVView (csv module, HttpResponse text/csv, monthly
statement) and ExportarPDFView using the GeraPDFMixin (template extrato_pdf.html).
Both filtered by month/year and by request.user. Add buttons on the dashboard.
```

M6 — API DRF + Token

```
Implement the api app: serializers for Transacao, Categoria, MetaEconomia.
ModelViewSets with authentication_classes=[TokenAuthentication],
permission_classes=[IsAuthenticated], and get_queryset filtered by request.user
(perform_create sets usuario=request.user). Register a DRF router. Add
obtain_auth_token endpoint at /api/token/. Add rest_framework and
rest_framework.authtoken to INSTALLED_APPS, run migrate. Show me curl examples to
get a token and list transactions.
```

M7 — testes

```
Write tests.py: (1) creating a SAÍDA above balance is rejected; (2) a user cannot
see another user's transactions (get_queryset isolation); (3) the API returns 401
without a token and 200 with a valid token. Keep them runnable with `manage.py test`.
```

Boas práticas durante o uso

- Um milestone por vez, **commit a cada milestone** (mensagens claras = prova de autoria na arguição).
- Sempre rode `manage.py runserver` e teste manualmente antes de commitar.
- Se o Claude propuser refatoração grande, peça pra explicar antes — vocês precisam entender o código pra arguição.

Prompt — Claude Design (3 telas faltantes)

> Cole no Claude Design, **no mesmo projeto** onde já foram geradas as 9 telas + `styles.css`. Objetivo: gerar APENAS os 3 templates que faltam, 100% consistentes com o que já existe.

```
You already generated the ACashController screens (base.html, login, cadastro,
dashboard, transacoes, transacao_form, categorias, categoria_form, metas) and a
shared styles.css. Now generate the 3 MISSING screens. Be STRICTLY CONSISTENT with
everything already created — same design system, same markup patterns, same CSS
classes, same Django comment style. DO NOT invent new CSS classes or restyle
anything; reuse the existing styles.css.

CONSISTENCY RULES (must follow exactly — these match the existing screens):
- Reuse styles.css as-is. Allowed classes only: .navbar/.navbar-brand/.logo-mark/
  .navbar-nav (a.active)/.navbar-end/.avatar/.navbar-username/.navbar-sair,
  .page/.page-header/.page-title, .card, .form-group/.form-label/.form-control
  (.is-invalid)/.form-hint/.form-row, .alert-inline, .form-actions,
  .btn/.btn-primary/.btn-secondary/.btn-outline/.btn-danger/.btn-inline/.btn-sm,
  .badge/.badge-entrada/.badge-saida, .table-container/.table-scroll, .td-muted.
- Design system: green #1D9E75 (primary), blue #2E6FB7 (accent), red/expense
  #C25E3A, font Poppins, CSS vars (--green/--blue/--red/--text/--text-muted/--border/
  --radius/--surface). Use var(--red) for destructive accents.
- Every screen (except the PDF one) extends base.html and includes the SAME navbar
  block + flash messages block as the other screens, with the correct nav item set
  to class="active". Keep the dual markup style: static preview HTML PLUS
  "<!-- Django: ... -->" comments showing the real template tags ({% extends %},
  {% block title %}, {% block content %}, {% csrf_token %}, {% url %}, {{ ... }}).
- pt-BR for all user-facing text.

SCREEN 1 — transacao_confirm_delete.html (DeleteView confirmation)
- Extends base.html, title "Excluir Transação — ACashController", nav "Transações"
  active.
- Centered narrow .card (max-width ~480px) inside .page, with a .page-header
  ("Excluir Transação" + a "← Voltar" .btn .btn-outline .btn-sm pointing to
  transacoes).
- Show the transaction being deleted using the SAME visual language as
  transacoes.html rows: descrição (bold), categoria (.td-muted), a .badge
  (badge-entrada/badge-saida) for tipo, the date (.td-muted), and the value using
  .td-val-saida / .td-val-entrada. Use static example data + Django comments
  ({{ object.descricao }}, {{ object.categoria.nome }}, {{ object.data|date:"d/m/Y" }},
  {{ object.valor|floatformat:2 }}, {{ object.tipo_display }} — remember tipo is a
  @property read from the category, there is NO tipo field).
- A confirmation question: "Tem certeza que deseja excluir esta transação? Esta
  ação não pode ser desfeita."
- A POST form: <!-- Django: <form method="post" action="{% url 'transacao_excluir'
  object.pk %}">{% csrf_token %} --> with .form-actions containing a
  .btn.btn-danger "Excluir" (submit) and a .btn.btn-outline "Cancelar" linking to
  transacoes. Static preview action="#".

SCREEN 2 — meta_form.html (create/edit a savings goal)
- Extends base.html, title "{% if form.instance.pk %}Editar{% else %}Nova{% endif %}
  Meta — ACashController", nav "Metas" active.
- Same layout pattern as categoria_form.html: a max-width ~520px wrapper, a
  .page-header (dynamic title + "← Voltar" .btn-outline .btn-sm to metas) and a
  .card holding the form.
- Include the validation error block exactly like categoria_form.html:
  <!-- Django: {% if form.errors %} --> .alert-inline with ⚠ ... <!-- {% endif %} -->
- Form fields matching the MetaEconomia model (field names → ids generated as
  id_<name>):
```

```
    * Nome (required) — id_nome, name="nome", maxlength="120", placeholder
      "Ex.: Fundo de Emergência, Viagem...". This is the model field `nome`
      (NOT titulo).
    * Valor Alvo (required) — id_valor_alvo, name="valor_alvo", type="number"
      step="0.01" min="0", placeholder "20000.00". Prefix hint "R$".
    * Valor Atual — id_valor_atual, name="valor_atual", type="number" step="0.01"
      min="0", default value 0, hint "Quanto você já juntou".
    * Prazo (opcional) — id_prazo, name="prazo", type="date".
  Use the .form-row (two-column) grid for Valor Alvo + Valor Atual, like the
  reference layout, collapsing on mobile (already handled by CSS).
- Per-field error hints in red exactly like categoria_form.html
  ({% if form.<campo>.errors %}<span class="form-hint" style="color:var(--red);">
  {{ form.<campo>.errors|join:", " }}</span>{% endif %}).
- .form-actions: .btn.btn-primary submit ("{% if form.instance.pk %}Salvar
  Alterações{% else %}Salvar Meta{% endif %}") + .btn.btn-outline "Cancelar" to
  metas. Static action="#"; Django comment shows the form posts to
  {% url 'meta_nova' %} / {% url 'meta_editar' meta.pk %} as appropriate.
- Optional: a tiny live progress preview (valor_atual / valor_alvo %) reusing the
  .goal-track/.goal-fill classes from metas.html, with minimal inline JS — same
  style as the color/badge preview JS in categoria_form.html (self-invoking
  function in {% block extra_js %}). Keep it optional and unobtrusive.

SCREEN 3 — extrato_pdf.html (monthly statement, rendered by xhtml2pdf/pisa — NOT a
browser screen)
- This is for server-side PDF generation with xhtml2pdf (pisa), so it must be a
  STANDALONE html (does NOT extend base.html, no navbar, no external CDN, no JS).
- IMPORTANT xhtml2pdf constraints: use simple inline CSS or a small <style> block
  with only well-supported properties (no flexbox/grid; use tables for layout, fixed
  widths, basic borders/padding/background-color, font-family). Keep it print-clean.
- Header: ACashController title + "AC" logo mark (simple colored box/table cell,
  green #1D9E75), the report title "Extrato Mensal", and the period
  ({{ mes_nome }} {{ ano }}) and the user ({{ usuario }} / {{ user.username }}).
- A summary table (Saldo, Total de Entradas, Total de Saídas) using the brand colors
  for the values (green entradas, red #C25E3A saídas, blue #2E6FB7 saldo).
- The main transactions table: columns Data | Descrição | Categoria | Tipo | Valor.
  Use a Django loop comment <!-- Django: {% for t in transacoes %} --> with one
  static example row per type. Entrada values in green with "+", Saída in red with
  "—". Right-align the value column. Add an {% empty %} comment row for "Nenhuma
  transação no período.".
- A footer line with generation date ({{ gerado_em|date:"d/m/Y H:i" }}) and a small
  "Gerado por ACashController" note.
- Use pt-BR currency formatting hints (R$, floatformat:2) in the Django comments.

OUTPUT: deliver the 3 HTML files named exactly transacao_confirm_delete.html,
meta_form.html, extrato_pdf.html. Keep them self-explanatory with the Django
comments so they can be dropped into the Django templates folders later.
```

Documentação Iterativa do Projeto

> Documento vivo. Atualizem a cada milestone concluído. Serve para que **os 3** entendam o projeto inteiro e consigam responder o professor (20 pts da arguição).
>
> Regra do grupo: ninguém commita uma feature sem registrar aqui **o que faz, onde está e por quê**.

Como funciona o projeto (visão de cima)

O usuário se cadastra e faz login. A partir daí, ele: 1. Cria **categorias** (Entrada ou Saída): Salário, Alimentação, Moradia… 2. Lança **transações** vinculadas a uma categoria. Saídas só passam se houver **saldo**. 3. Vê o **dashboard**: saldo, gráfico de linha (patrimônio mês a mês) e pizza (gastos por categoria). 4. Define **metas de economia** com barra de progresso. 5. Exporta o **extrato mensal** em CSV ou PDF. 6. (Opcional) Consome os dados via **API REST**, autenticando com **token**.

Tudo é isolado por usuário: ninguém vê dado de ninguém.

Fluxo de uma requisição (request → response)

```
Navegador
   │  GET /transacoes/
   ▼
core/urls.py  ──include──►  financas/urls.py  ──►  TransacaoListView (CBV)
   │
   ├─ LoginRequiredMixin  → exige usuário logado (senão redireciona p/ login)
   ├─ get_queryset()      → filtra Transacao.objects.filter(usuario=request.user)
   ▼
template financas/transacao_list.html  →  HTML renderizado
```

Para um POST de nova transação:

```
POST /transacoes/nova/ → TransacaoCreateView
   ├─ get_form_kwargs() injeta usuario=request.user no form
   ├─ form.clean()  → se categoria.tipo='S' e saldo < valor → ValidationError("Saldo insuficiente")
   ├─ form_valid()  → seta instance.usuario = request.user e salva
   ▼
redirect para a lista + messages.success
```

Decisões de arquitetura (e o porquê)

Decisão	Por quê (argumento na arguição)
CBV em tudo	Reuso e organização; mixins evitam repetição (DRY/SRP). Enunciado exige CBV.
Saldo derivado, não armazenado	Fonte única de verdade; impossível ficar inconsistente. Recalculado com `Sum`.
Validação no Form, não no Model	A regra depende do **usuário logado** (contexto da request), que o model não conhece. O form recebe `usuario` da view.

Decisão	Por quê (argumento na arguição)
Mixin `UsuarioQuerysetMixin`	Centraliza o isolamento por usuário em um lugar só (DRY).
`GeraPDFMixin` (xhtml2pdf)	Reaproveitado do repo de referência; gera PDF a partir de template HTML.
DRF para a API	Token auth pronto, serializers e viewsets reduzem boilerplate.
Chart.js no front	Gráficos leves; o Django só fornece o JSON, separando dados de apresentação.
Apps separados	Cada app tem uma responsabilidade (SRP): `financas` (domínio), `dashboard` (relatórios), `api` (integração), `usuario` (auth).

Mapa: requisito do enunciado → arquivo/linha

Requisito	Onde está
CBV de lançamento categorizado	`financas/views.py → TransacaoCreateView`
Validação de saldo	`financas/forms.py → TransacaoForm.clean()` + `financas/services.py → saldo_usuario()`
Gráfico de linha (patrimônio)	`dashboard/views.py → DashboardView` + `static/js/charts.js`
Gráfico de pizza (categorias)	idem
Exportar CSV	`dashboard/views.py → ExportarCSVView`
Exportar PDF	`dashboard/views.py → ExportarPDFView` + `utils/my_mixins.py → GeraPDFMixin`
API de transações	`api/views.py → TransacaoViewSet` + `api/serializers.py`
Token Authentication	`api/views.py (authentication_classes)` + `/api/token/`
Isolamento por usuário (web)	`LoginRequiredMixin` + `get_queryset()` em `financas/views.py`
Isolamento por usuário (API)	`TransacaoViewSet.get_queryset()` filtra por `request.user`

> **Atualizem a coluna "Onde está" com os nomes/linhas reais** conforme implementam.

Glossário rápido (pra não travar na pergunta)

- **CBV (Class-Based View):** view escrita como classe; herda comportamento de classes prontas do Django (ListView, CreateView…).
- **Mixin:** classe pequena com um comportamento isolado que você "mistura" em outras classes via herança múltipla.
- **`get_queryset()`:** método que define **quais objetos** a view enxerga. Filtramos por `request.user`.
- **`LoginRequiredMixin`:** bloqueia a view para quem não está logado.
- **Serializer (DRF):** converte objetos do model em JSON (e valida JSON de entrada).
- **TokenAuthentication:** cada usuário tem um token; a API exige `Authorization: Token <chave>`.
- **`TruncMonth` / `Sum`:** funções de agregação do ORM para somar valores por mês.

Log de progresso (preencher por milestone)

Data	Milestone	Quem	Notas / decisões
	M1 modelos		
	M2 auth/isolamento		
	M3 CRUD + saldo		
	M4 dashboard		
	M5 export CSV/PDF		
	M6 API DRF		
	M7 testes		

Banco de Perguntas do Professor (20 pts) — com respostas

> Simulem isto antes do dia 11/06. Cada integrante deve saber responder **todas**, não só a sua parte.

Bloco 1 — CBV e arquitetura

P: Por que vocês usaram Class-Based Views em vez de function views? R: CBVs trazem comportamento pronto (ListView, CreateView, etc.), reduzem repetição e permitem reutilizar regras via **mixins**. O enunciado também exige CBV. Ex.: nossa `TransacaoCreateView` herda de `CreateView` e só sobrescrevemos `get_form_kwargs()` e `form_valid()`.

P: O que é um mixin? Onde vocês usaram? R: É uma classe pequena com um comportamento isolado, "misturada" por herança múltipla. Usamos `UsuarioQuerysetMixin` (isola dados por usuário) e `GeraPDFMixin` (gera PDF). Vantagem: o mesmo código serve para várias views (DRY).

P: Por que vários apps Django? R: Separação de responsabilidades (SRP): `usuario` (auth), `financas` (domínio), `dashboard` (relatórios/gráficos), `api` (integração REST). Facilita manutenção e teste.

Bloco 2 — Validação de saldo (provável foco)

P: Onde e como vocês validam se há saldo suficiente? R: No método `clean()` do `TransacaoForm`. O tipo (E/S) **não fica na transação** — é derivado da `categoria` escolhida. Quando `categoria.tipo == 'S'`, comparamos `saldo_usuario(user)` com o valor; se for menor, levantamos `ValidationError("Saldo insuficiente")`. O formulário recebe o `usuario` da view via `get_form_kwargs()`.

P: Por que validar no Form e não no Model? R: A regra depende do **usuário logado**, que é contexto da requisição. O Model não conhece a request; o Form/View conhece. Colocar no Form mantém a regra perto do contexto certo e exibe o erro na tela corretamente.

P: O saldo fica salvo no banco? R: Não. É **derivado**: `soma(Entradas) - soma(Saídas)` do usuário, calculado com `aggregate(Sum('valor'))`. Assim nunca fica inconsistente — a fonte da verdade são as transações.

P: E se duas saídas forem feitas ao mesmo tempo? R: Para o escopo acadêmico, validamos por request. (Se quiser blindar concorrência: usar `select_for_update()` numa transação atômica — bom mencionar como melhoria futura.)

Bloco 3 — Isolamento por usuário

P: Como garantem que um usuário não vê dados de outro? R: Duas camadas: `LoginRequiredMixin` (exige login) e `get_queryset()` filtrando por `request.user` em **todas** as views privadas. Na API, o `get_queryset()` do ViewSet também filtra por `request.user`.

P: O que acontece se eu trocar o ID na URL para acessar a transação de outro usuário? R: Não aparece. Como o `get_queryset()` já filtra por `request.user`, o objeto de outro usuário simplesmente não está no queryset → retorna 404.

P: Onde definem para onde vai um usuário não logado? R: `LOGIN_URL` no settings; o `LoginRequiredMixin` redireciona para lá.

Bloco 4 — Dashboard e gráficos

P: Como os dados chegam ao Chart.js? R: A `DashboardView` calcula as séries no backend (ORM, com `TruncMonth` + `Sum`), passa como JSON ao template (json_script), e o `charts.js` lê esse JSON e desenha. O Django cuida dos **dados**; o Chart.js, da **apresentação**.

P: Como calculam a evolução mensal do patrimônio? R: Agrupamos transações por mês (`TruncMonth`), somamos entradas e saídas de cada mês e **acumulamos** (entradas − saídas) mês a mês → patrimônio acumulado.

P: E o gráfico de pizza? R: `Transacao` do tipo Saída, agrupado por categoria (`values('categoria__nome').annotate(Sum('valor'))`). Cada fatia usa a cor da categoria (`Categoria.cor`).

Bloco 5 — Exportação CSV/PDF

P: Como geram o CSV? R: Módulo `csv` do Python escrevendo num `HttpResponse` com `content_type='text/csv'` e header `Content-Disposition: attachment`. Filtrado por mês/ano e por usuário.

P: E o PDF? R: `GeraPDFMixin` (xhtml2pdf/pisa): renderiza um template HTML (`extrato_pdf.html`) e converte para PDF. Mesmo padrão do repo de referência.

Bloco 6 — API e Token Authentication

P: Como funciona a autenticação da API? R: DRF `TokenAuthentication`. Cada usuário tem um token (tabela `authtoken`). O cliente envia `Authorization: Token <chave>`. Sem token válido → 401. O token é obtido no endpoint `/api/token/` (`obtain_auth_token`) enviando usuário e senha.

P: Qual a diferença entre Token e Session auth? R: Session usa cookie e é pensada para o navegador; Token é stateless, ideal para clientes externos (apps, scripts). Usamos Session no site e Token na API.

P: A API também isola por usuário? R: Sim. O `get_queryset()` do ViewSet filtra por `request.user`, e `perform_create()` seta `usuario=request.user`. Permissão `IsAuthenticated`.

P: Mostre como pegar um token. R:

```
curl -X POST http://localhost:8000/api/token/ \
  -d "username=ana&password=senha"
# → {"token":"abc123..."}
curl http://localhost:8000/api/transacoes/ \
  -H "Authorization: Token abc123..."
```

Bloco 7 — Banco, models e ORM

P: Por que `DecimalField` para valor e não Float? R: Dinheiro exige precisão exata; Float tem erro de ponto flutuante. `DecimalField` evita isso.

P: Para que serve o `related_name`? R: Permite acessar o lado inverso da relação: `user.transacoes.all()`, `user.metas.all()`.

P: O que é uma migração? R: Arquivo que descreve mudanças no schema do banco. `makemigrations` gera; `migrate` aplica.

P: Por que `on_delete=PROTECT` na categoria da transação? R: Impede apagar uma categoria que ainda tem transações, evitando perda de histórico.

Bloco 8 — Pegadinhas comuns

- **"Quem escreveu essa parte?"** → todos sabem explicar tudo (combinar antes quem fala o quê, mas qualquer um responde).
- **"Mostra rodando."** → tenham o servidor pronto, um usuário com dados de exemplo e os gráficos populados.
- **"E se eu logar com outro usuário?"** → demonstrar que os dados mudam (isolamento).
- **"Cadê o commit dessa feature?"** → histórico do Git limpo, um commit por milestone.

Checklist final antes da apresentação

- [ ] Servidor roda sem erro (`manage.py runserver`).
- [ ] 2 usuários com dados → provar isolamento ao vivo.
- [ ] Dashboard com os 2 gráficos populados.
- [ ] Exportar CSV e PDF funcionando.
- [ ] API: obter token + listar transações no Postman/curl.
- [ ] Tentar lançar Saída sem saldo → erro aparece.
- [ ] Cada integrante respondeu o banco de perguntas em simulação.
- [ ] README com passos de instalação.

Deploy e Variáveis de Ambiente — ACashController

> Hospedagem escolhida: **PythonAnywhere** (free tier), mantendo **SQLite**. Grátis, disco persistente (dados não somem), feito pra Django acadêmico. URL pública é desejável mas **não obrigatória** — serve só pra mostrar.

Parte 1 — Variáveis de ambiente (`django-environ`)

Mesmo padrão do repo de referência (`ecommerce2026` usa `.env` + `.env_vazio`). Objetivo: **nada de segredo no código nem no GitHub**.

1.1 Instalar

```
pip install django-environ
```

Adicionar ao `requirements.txt` :

```
django-environ==0.11.2
```

1.2 `.gitignore` (na raiz)

```
.env
db.sqlite3
__pycache__/
*.pyc
/staticfiles/
```

> **Nunca** commitar `.env` nem `db.sqlite3` .

1.3 `.env.example` (ESTE sim vai pro Git — é só o modelo, sem valores reais)

```
SECRET_KEY=troque-por-uma-chave-secreta
DEBUG=True
ALLOWED_HOSTS=127.0.0.1,localhost
DATABASE_URL=sqlite:///db.sqlite3
```

1.4 `.env` (cada integrante cria o seu localmente, copiando do .example)

```
SECRET_KEY=django-insecure-COLOQUE-ALGO-LONGO-E-ALEATORIO
DEBUG=True
ALLOWED_HOSTS=127.0.0.1,localhost,.app.github.dev
DATABASE_URL=sqlite:///db.sqlite3
```

> Gerar uma SECRET_KEY: `python -c "from django.core.management.utils import get_random_secret_key;`

```
print(get_random_secret_key())"
```

1.5 `core/settings.py`

```
from pathlib import Path
import environ

BASE_DIR = Path(__file__).resolve().parent.parent

env = environ.Env(DEBUG=(bool, False))
environ.Env.read_env(BASE_DIR / '.env')

SECRET_KEY = env('SECRET_KEY')
DEBUG = env('DEBUG')
ALLOWED_HOSTS = env.list('ALLOWED_HOSTS', default=['127.0.0.1', 'localhost'])

DATABASES = {
    'default': env.db('DATABASE_URL', default=f'sqlite:///{BASE_DIR / "db.sqlite3"}')
}

# Arquivos estáticos (necessário pro deploy)
STATIC_URL = 'static/'
STATIC_ROOT = BASE_DIR / 'staticfiles'      # destino do collectstatic
STATICFILES_DIRS = [BASE_DIR / 'static']    # seus css/js de dev
```

> **Ponto de arguição:** "Por que `.env`?" → para não versionar segredos (SECRET_KEY), permitir config diferente por ambiente (dev vs prod) sem mudar código, e seguir o padrão 12-factor.

Parte 2 — Deploy no PythonAnywhere (passo a passo)

> Plano **Beginner é grátis**. Roda Django + SQLite com disco persistente. Dá uma URL tipo `https://SEUUSUARIO.pythonanywhere.com`.

2.1 Pré-requisitos

- Projeto já no GitHub (repo do grupo).
- Conta grátis em https://www.pythonanywhere.com (Beginner).

2.2 Subir o código (console Bash do PythonAnywhere)

No dashboard → **Consoles** → **Bash**:

```
git clone https://github.com/SEU-USUARIO/SEU-REPO.git
cd SEU-REPO

# ambiente virtual
python3.10 -m venv venv
source venv/bin/activate
pip install -r requirements.txt
```

2.3 Criar o `.env` no servidor

O `.env` não veio do Git (está no .gitignore), então criar lá:

```
nano .env
```

Conteúdo (ATENÇÃO: DEBUG=False e o host do PythonAnywhere):

```
SECRET_KEY=uma-chave-secreta-bem-longa-de-producao
DEBUG=False
ALLOWED_HOSTS=SEUUSUARIO.pythonanywhere.com
DATABASE_URL=sqlite:///db.sqlite3
```

2.4 Migrações, superuser e estáticos

```
python manage.py migrate
python manage.py createsuperuser
python manage.py collectstatic --noinput
# (opcional) rodar o seed de categorias:
python manage.py seed_categorias
```

2.5 Criar a Web App

No dashboard → aba **Web** → **Add a new web app**: 1. Escolher **Manual configuration** (não o "Django" automático) → Python 3.10. 2. Na seção **Virtualenv**, apontar para: `/home/SEUUSUARIO/SEU-REPO/venv` 3. Editar o **WSGI configuration file** (link na aba Web) e deixar assim:

```
import os, sys

path = '/home/SEUUSUARIO/SEU-REPO'
if path not in sys.path:
    sys.path.insert(0, path)

os.environ['DJANGO_SETTINGS_MODULE'] = 'core.settings'

from django.core.wsgi import get_wsgi_application
application = get_wsgi_application()
```

1. Na seção **Static files** da aba Web, mapear: | URL | Directory | |---|---| | `/static/` | `/home/SEUUSUARIO/SEU-REPO/staticfiles` |
2. Clicar em **Reload** (botão verde no topo da aba Web).

Pronto → `https://SEUUSUARIO.pythonanywhere.com`

2.6 Atualizar depois de mudanças (workflow de re-deploy)

Toda vez que vocês fizerem push no GitHub:

```
# no console Bash do PythonAnywhere
cd ~/SEU-REPO
source venv/bin/activate
git pull
pip install -r requirements.txt        # só se mudou dependência
python manage.py migrate               # só se mudou model
python manage.py collectstatic --noinput
```

Depois → aba **Web** → botão **Reload**.

> É "manual", mas são 4 comandos + 1 clique. Os **dados no SQLite persistem** entre deploys (diferente do Render).

Parte 3 — Checklist de segurança pra produção

- [ ] `DEBUG=False` no `.env` do servidor.
- [ ] `ALLOWED_HOSTS` com o domínio do PythonAnywhere.
- [ ] `SECRET_KEY` de produção diferente da de dev.
- [ ] `.env` e `db.sqlite3` no `.gitignore` (confirmar que NÃO foram commitados).
- [ ] `collectstatic` rodado (senão o CSS/JS não carrega com DEBUG=False).
- [ ] (Opcional) `CSRF_TRUSTED_ORIGINS = ['https://SEUUSUARIO.pythonanywhere.com']` no settings.

Parte 4 — Resumo dev vs prod

	Desenvolvimento (Codespace/local)	Produção (PythonAnywhere)
`DEBUG`	`True`	`False`
`ALLOWED_HOSTS`	localhost, `.app.github.dev`	`SEUUSUARIO.pythonanywhere.com`
Banco	SQLite (`db.sqlite3`)	SQLite (`db.sqlite3`) — persistente
Estáticos	servidos pelo runserver	`collectstatic` + mapeamento na aba Web
Servidor	`manage.py runserver`	WSGI do PythonAnywhere

> O código é o **mesmo**; só o `.env` muda entre ambientes. Esse é o ponto-chave.